

Transistor Switching Network Synthesis

Modeling, State Representation, and Generation Process

A formal introduction to switching networks,
Boolean function representation, and the sequential
generation MDP with SOP-based state initialization.

TransNetGame Project

Contents

1	Switching Networks	2
1.1	Basic Definitions	2
1.2	Correctness and Safety	2
1.3	Monotonicity A Key Structural Property	2
2	SOP Representation and the Series-Parallel Network	3
2.1	Sum-of-Products (SOP)	3
2.2	Series Chain per Minterm	3
2.3	Parallel Combination: the SOP Series-Parallel Network	3
2.4	Illustrated Example: XOR ($N = 2$)	3
3	Sequential Generation as a Markov Decision Process	4
3.1	Two-Component State	4
3.2	The Compensate Network: Always the Full SOP	4
3.3	Action Space	5
3.4	Transition	6
3.5	Termination	6
4	Policy Parameterization and Imitation Pretraining	7
4.1	Factorized Policy	7
4.2	Prefix Supervision	7
5	Network Reduction	7
6	Reinforcement Learning Formulation	8
6.1	Shaped Coverage Reward	8
6.2	Advantage Estimation	8
6.3	Escaping Converged Local Optima	9
7	Experiments	9
7.1	Setup	9
7.2	Construction Order Dominates Architecture	9
7.3	Label Consistency	9
7.4	Transistor-Count Optimization	10
7.5	Diagnostic Analyses	10
8	Planned Experiments	10

1. Switching Networks

1.1 Basic Definitions

We work with N Boolean input variables $\mathbf{x} = (x_1, \dots, x_N)$. An input pattern $\pi \in \{0, 1\}^N$ assigns a concrete value to each variable. The **literal alphabet** is

$$\mathcal{L} = \{x_1, \neg x_1, x_2, \neg x_2, \dots, x_N, \neg x_N\}, \quad |\mathcal{L}| = 2N.$$

Literal x_j evaluates to π_j under π ; literal $\neg x_j$ evaluates to $1 - \pi_j$.

Definition 1.1 (Switching Network). A **switching network** over variables $\mathbf{x}$ is an undirected multigraph

$$G = (V, E), \quad V \supseteq \{\mathbf{S}, \mathbf{T}\}, \quad E \subseteq \binom{V}{2} \times \mathcal{L},$$

where $\mathbf{S}$ (source) and $\mathbf{T}$ (drain/sink) are two distinguished terminal nodes. Every edge $(u, v, \ell) \in E$ represents a **transistor** with control literal $\ell \in \mathcal{L}$.

Definition 1.2 (Conduction). Under input pattern π , transistor (u, v, ℓ) **conducts** if and only if $\ell(\pi) = 1$. Network G **conducts** $\mathbf{S} \rightarrow \mathbf{T}$ under π if and only if there exists a path from $\mathbf{S}$ to $\mathbf{T}$ in the subgraph of G restricted to conducting transistors.

1.2 Correctness and Safety

Given a Boolean function $f : \{0, 1\}^N \rightarrow \{0, 1\}$, partition all 2^N input patterns into:

$$P^+ = \{\pi : f(\pi) = 1\} \quad (\text{on-patterns}), \quad P^- = \{\pi : f(\pi) = 0\} \quad (\text{off-patterns}).$$

Definition 1.3 (Correct Implementation). Network G **correctly implements** f if and only if:

$$\begin{aligned} \forall \pi \in P^+ : \quad & \mathbf{S} \rightarrow \mathbf{T} \text{ conducts in } G \text{ under } \pi, & (\text{Correctness}) \\ \forall \pi \in P^- : \quad & \mathbf{S} \not\rightarrow \mathbf{T} \text{ in } G \text{ under } \pi. & (\text{Safety}) \end{aligned}$$

The synthesis goal is to find a correct implementation with the minimum number of transistors:

$$\min\{|E| : G = (V, E) \text{ correctly implements } f\}.$$

1.3 Monotonicity A Key Structural Property

Theorem 1.4 (Monotonicity of Conducting Paths). *For any network G and any transistor $e \notin E$,*

$$\text{Paths}(G \cup \{e\}) \supseteq \text{Paths}(G),$$

where $\text{Paths}(H)$ denotes the set of $(\mathbf{S}, \mathbf{T})$ conducting paths of H under all patterns.

Proof. Adding a transistor can activate new paths but cannot deactivate any existing conducting transistor. Hence every path that existed in G still exists in $G \cup \{e\}$. $\square$

Corollary 1.5 (Inheritable Safety). *If G is safe (no $\mathbf{S} \rightarrow \mathbf{T}$ path under any $\pi \in P^-$), then $G \cup \{(u, v, \ell)\}$ is safe if and only if (u, v, ℓ) does not create any $\mathbf{S} \rightarrow \mathbf{T}$ path under any $\pi \in P^-$.*

Corollary 1.5 means safety is an *incrementally checkable invariant*: we need only verify the new transistor, not the entire network.

2. SOP Representation and the Series-Parallel Network

2.1 Sum-of-Products (SOP)

Any Boolean function f can be expressed as a disjunction of **minterms**. For N variables, each on-pattern $\pi = (b_1, \dots, b_N) \in P^+$ uniquely corresponds to the product term

$$T_\pi = \ell_1^\pi \cdot \ell_2^\pi \cdots \ell_N^\pi, \quad \ell_j^\pi = \begin{cases} x_j & \text{if } b_j = 1, \\ \neg x_j & \text{if } b_j = 0. \end{cases}$$

The full SOP is $f = \bigvee_{\pi \in P^+} T_\pi$.

2.2 Series Chain per Minterm

Product term T_π maps directly to a **series chain** of N transistors. Introduce $N - 1$ fresh internal nodes $c_{\pi,1}, \dots, c_{\pi,N-1}$ and connect:

$$\mathbf{S} \xrightarrow{\ell_1^\pi} c_{\pi,1} \xrightarrow{\ell_2^\pi} c_{\pi,2} \cdots c_{\pi,N-1} \xrightarrow{\ell_N^\pi} \mathbf{T}.$$

This chain conducts under pattern π' if and only if every literal ℓ_j^π evaluates to 1 under π' , which happens precisely when $\pi' = \pi$.

2.3 Parallel Combination: the SOP Series-Parallel Network

Definition 2.1 (SOP Network). The **SOP series-parallel network** for f is the union of all minterm chains sharing terminal nodes $\mathbf{S}$ and $\mathbf{T}$:

$$G_{\text{SOP}}(f) = \left(\{\mathbf{S}, \mathbf{T}\} \cup \{c_{\pi,j} : \pi \in P^+, j = 1, \dots, N-1\}, \bigcup_{\pi \in P^+} \text{chain}(\pi) \right).$$

Theorem 2.2 (Correctness and Safety of G_{SOP}). $G_{\text{SOP}}(f)$ correctly implements f .

Proof. Correctness. For $\pi \in P^+$, every literal ℓ_j^π evaluates to 1 under π , so the chain for π is fully conducting: $\mathbf{S} \rightarrow \mathbf{T}$.

Safety. For $\pi' \in P^-$, consider any chain indexed by $\pi \in P^+$. Since $\pi' \neq \pi$, they differ in at least one bit position j^* . Then $\ell_{j^*}^\pi(\pi') = 0$: the j^* -th transistor of the π -chain is off. Every chain contains at least one non-conducting transistor under π' , so no $\mathbf{S} \rightarrow \mathbf{T}$ path exists. $\square$

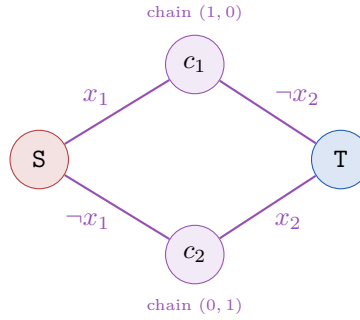
The SOP network is always *correct and safe by construction*, but is generally *not minimal*.

Quantity	Value
Transistors $ E_{\text{SOP}} $	$ P^+ \times N$
Internal nodes	$ P^+ \times (N - 1)$

2.4 Illustrated Example: XOR ($N = 2$)

$$f(x_1, x_2) = x_1 \oplus x_2, \quad P^+ = \{(1, 0), (0, 1)\}, \quad P^- = \{(0, 0), (1, 1)\}.$$

G_{SOP} consists of two series chains in parallel:



$|E_{\text{SOP}}| = 4$ transistors, $|P^+| \times N = 2 \times 2 = 4$. (Already optimal for XOR with $N = 2$.)

3. Sequential Generation as a Markov Decision Process

We model transistor network synthesis as a sequential decision process. At each step the agent places one transistor; the episode terminates when the synthesized network already covers all on-patterns.

3.1 Two-Component State

The state at every time step t consists of two disjoint subnetworks that together form the **unified graph** GC_t :

Component	Definition	Role
Synthesized network G_t	$(V_{G,t}, E_{G,t}); V_{G,0} = \{\mathbf{S}, \mathbf{T}\}, E_{G,0} = \emptyset$	Transistors placed by the agent so far
Compensate network C_t	$G_{\text{SOP}}(f); \text{ fixed for all } t$	Full SOP network for f ; encodes all on-patterns
Unified graph GC_t	$G_t \cup C_t = (V_{G,t} \cup V_C, E_{G,t} \cup E_C)$	Full observation available to the policy

$\mathbf{S}$ and $\mathbf{T}$ are shared between both components. Internal nodes of G_t and C_t are disjoint sets.

3.2 The Compensate Network: Always the Full SOP

At every step t , the compensate network is the *complete, fixed* SOP series-parallel network for the entire Boolean function f :

Definition 3.1 (Compensate Network). The compensate network is the full SOP series-parallel network for f and is *fixed throughout the entire episode*:

$$C_t = G_{\text{SOP}}(P^+) = G_{\text{SOP}}(f) \quad \text{for all } t \geq 0.$$

It contains one series chain per on-pattern in P^+ , and all chains share the terminals $\mathbf{S}$ and $\mathbf{T}$.

Why always the full SOP?

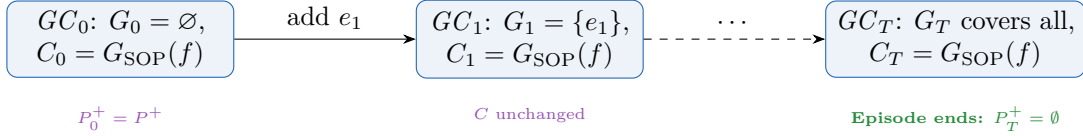
The compensate network is a *fixed reference* that encodes the complete specification of the Boolean function throughout the episode.

- **Information completeness:** C_t always encodes *every* on-pattern as an explicit series

chain; the policy always has full knowledge of the function to be synthesized.

- **Structural stability:** The same series-parallel topology is present at every step, giving the policy a consistent graph structure regardless of how many transistors have been placed.
- **No Boolean minimization:** Building $G_{\text{SOP}}(f)$ requires only enumerating P^+ ; no external logic solver is needed.

The compensate network $C_t = G_{\text{SOP}}(f)$ remains unchanged from step 0 to termination. Coverage progress is tracked separately via $P_t^+ \subseteq P^+$ (on-patterns not yet routed in G_t). The episode ends when $P_t^+ = \emptyset$, i.e. G_t alone covers all on-patterns.



3.3 Action Space

Each action is a triple

$$a_t = (\text{src}, \text{drn}, \ell) \in V_{G,t} \times (V_{G,t} \cup \{n_{\text{new}}\}) \times \mathcal{L},$$

where n_{new} denotes a fresh internal node to be instantiated. The action is subject to two structural constraints enforced before any literal is selected:

Distinctness constraint. Transistors are undirected, so (u, v, ℓ) and (v, u, ℓ) denote the same device. At generation time the only structural requirement is that the two endpoints differ,

$$\text{valid_drn}(\text{src}) = (V_{G,t} \cup \{n_{\text{new}}\}) \setminus \{\text{src}\},$$

with the source additionally restricted to nodes that are reachable from **S** or **T** under at least one still-uncovered on-pattern (a node that no uncovered pattern can reach cannot contribute coverage). Duplicate orientations are eliminated not by an index rule but by the canonical *construction order* used to linearize reference networks during supervision, defined next.

Construction order (supervision linearization). Training supervision requires a canonical linearization $\ell_1, \dots, \ell_R$ of the R undirected transistors of a reference network into a sequence of MDP actions. We use a *path (ear) decomposition*: the sequence is generated by successive walks, where the first walk starts at **S** and extends through unvisited nodes until it closes into already-visited structure (preferentially at **T**), and each subsequent walk (an *ear*) starts at an already-visited node and again extends until it reconnects. Formally, each emitted action $(\text{src}, \text{drn}, \ell)$ satisfies the invariant that src is already present in the partial network, so the walk direction not a node-index comparison fixes the orientation.

This ordering is not merely a tie-breaking convention: it aligns the supervision with the semantics of the task. Because conduction is only achieved when a $\text{S} \rightarrow \text{T}$ path *closes*, decomposing the target into completed paths means each label continues a coherent sub-goal. A further consequence is structural: during a walk extension the partial network contains exactly one dangling endpoint, and that endpoint *is* the next source, so the src decision is determined by the graph rather than by an arbitrary indexing convention. The residual difficulty is concentrated at *ear starts*, where the model must decide from which existing node to branch the next path which is precisely the structure-sharing decision that governs network compactness. Section 7 quantifies

the effect: replacing an index-based breadth-first linearization with this path decomposition raises source-selection accuracy from 0.63 to 0.93 on 4-input functions and from 0.50 to 0.91 on 6-input functions, at identical model capacity.

Safety mask (BFS cutoff). Once (src, drn) is fixed, any literal whose addition would create a $\mathbf{S} \rightarrow \mathbf{T}$ path under any $\pi \in P^-$ is forbidden:

$$\mathcal{F}(\text{src}, \text{drn}) = \{\ell \in \mathcal{L} : \exists \pi \in P^-, \mathbf{S} \rightarrow \mathbf{T} \text{ in } (V_{G,t}, E_{G,t} \cup \{(\text{src}, \text{drn}, \ell)\}) \text{ under } \pi\}.$$

Only literals in $\mathcal{L} \setminus \mathcal{F}(\text{src}, \text{drn})$ are allowed.

3.4 Transition

Given action $a_t = (\text{src}, \text{drn}, \ell)$:

1. **Update synthesized network.** If $\text{drn} = n_{\text{new}}$, create a new node n_k and set $\text{drn} \leftarrow n_k$:

$$E_{G,t+1} = E_{G,t} \cup \{(\text{src}, \text{drn}, \ell)\}, \quad V_{G,t+1} = V_{G,t} \cup \{\text{drn}\}.$$

2. **Coverage check on G_{t+1} only.** Find newly covered on-patterns:

$$\Delta_t = \{\pi \in P_t^+ : \mathbf{S} \rightarrow \mathbf{T} \text{ in } G_{t+1} \text{ under } \pi\}.$$

3. **Update coverage tracking.** Record newly covered on-patterns:

$$P_{t+1}^+ = P_t^+ \setminus \Delta_t.$$

The compensate network is *not* modified: $C_{t+1} = G_{\text{SOP}}(f)$.

4. **New unified state.** $GC_{t+1} = G_{t+1} \cup C_{t+1}$.

Implementation note. The BFS safety check (step 2 of the action selection) must use $E_{G,t}$ only not the combined $E_{G,t} \cup E_{C,t}$. Compensate transistors represent coverage obligations, not physical switches in the circuit. The implementation must therefore maintain $E_{G,t}$ and $E_{C,t}$ as separate edge sets, even though GC_t presents them as a single unified graph to the policy.

3.5 Termination

The episode terminates when $P_t^+ = \emptyset$, i.e. when G_t routes $\mathbf{S} \rightarrow \mathbf{T}$ under every $\pi \in P^+$. The compensate network $C_t = G_{\text{SOP}}(f)$ is present but no longer contributes new obligations. At that point:

- **Correctness:** G_T routes $\mathbf{S} \rightarrow \mathbf{T}$ under every $\pi \in P^+$ (since $P_T^+ = \emptyset$ means all patterns were covered during the episode).
- **Safety:** G_T never routes $\mathbf{S} \rightarrow \mathbf{T}$ under any $\pi \in P^-$ (guaranteed by the BFS mask at every step, by induction from Corollary 1.5).

No explicit “stop” action is required.

4. Policy Parameterization and Imitation Pretraining

4.1 Factorized Policy

The unified state GC_t is encoded by a message-passing network, producing a node embedding $h_v \in \mathbb{R}^d$ for every $v \in GC_t$ and a graph embedding $g \in \mathbb{R}^{2d}$. The virtual node n_{new} is represented by a learned embedding $h_{n_{\text{new}}}$, so that creating a fresh internal node is an ordinary choice in the same action space.

An action is emitted by four heads applied in sequence, each conditioned on the previous choices:

$$\pi_\theta(a_t \mid GC_t) = \underbrace{\pi^{\text{src}}}_{\text{which node}} \cdot \underbrace{\pi^{\text{drn}}}_{\text{which partner}} \cdot \underbrace{\pi^{\text{var}}}_{\text{which variable}} \cdot \underbrace{\pi^{\text{sgn}}}_{\text{polarity}}.$$

The source head scores every candidate node, the drain head scores candidates conditioned on the chosen source, and the variable and polarity heads select the literal ℓ conditioned on both endpoints. Structural masks (distinctness, reachability) and the safety mask $\mathcal{F}$ of the action space are applied as $-\infty$ logit offsets before each softmax, so every sampled action is admissible by construction and the safety invariant of Corollary 1.5 holds for all rollouts.

4.2 Prefix Supervision

Given a reference network linearized as $\ell_1, \dots, \ell_R$ by the path decomposition of Section 3.3, every prefix $k = 0, \dots, R - 1$ yields one training example: the input is the state GC_k induced by the first k transistors together with the full compensate network, and the target is the action ℓ_{k+1} . A network with R transistors therefore contributes R supervised decisions. Training minimizes the factorized negative log-likelihood

$$\mathcal{L}(\theta) = -\sum_k \left[w_{\text{src}} \log \pi_\theta^{\text{src}} + \log \pi_\theta^{\text{drn}} + \log \pi_\theta^{\text{var}} + \log \pi_\theta^{\text{sgn}} \right],$$

where w_{src} optionally up-weights the source term, which empirically dominates the residual loss.

Label consistency. A Boolean function typically admits many distinct optimal or near-optimal networks. If several implementations of the *same* function are all retained as supervision, identical states are paired with conflicting targets and the objective acquires an irreducible floor. Retaining a single implementation per function (the most compact one) removes this conflict; Section 7 quantifies the effect.

5. Network Reduction

A generated network may contain transistors that do not affect the function it computes. Two nested notions of reduction are useful; both are applied before a network’s cost is reported.

Definition 5.1 (Topological reduction). A transistor is *structurally dead* if it lies on no simple $S \rightarrow T$ path. Such transistors cannot participate in any conduction path and may be deleted. The reduced network is obtained by keeping exactly those edges whose biconnected block lies on the block-cut-tree path between S and T .

Definition 5.2 (Functional reduction). A network is *irredundant* if no single transistor can be deleted while still covering every on-pattern. A functionally reduced network is obtained by deleting such transistors greedily until a fixed point.

Lemma 5.3 (Safety is preserved under deletion). *Let $G' \subseteq G$ be obtained by deleting transistors. If G is safe (routes $S \rightarrow T$ under no $\pi \in P^-$), then G' is safe.*

Proof. Every path in G' is a path in G . If G' conducted under some $\pi \in P^-$, the same path would conduct in G , contradicting safety of G . $\square$

Lemma 5.3 is what makes functional reduction cheap: deletion can only *reduce* conduction, so no off-pattern check is required and it suffices to re-verify on-pattern coverage. We write $T_{\text{eff}}(G)$ for the size of the reduced network and use it as the cost of G throughout, both when reporting results and inside the reinforcement-learning reward. Reporting unreduced sizes systematically overstates cost: on the 6-input benchmark of Section 7, functional reduction lowers the mean generated size from 41.4 to 19.0 transistors.

6. Reinforcement Learning Formulation

Imitation pretraining optimizes agreement with a particular reference construction, not the design objective itself. We therefore finetune the pretrained policy per target function with policy-gradient updates whose reward is the quantity we actually care about.

6.1 Shaped Coverage Reward

Terminal-only rewards are uninformative when the policy rarely completes a network: every rollout scores zero and no gradient survives. We therefore award partial credit for coverage. For a rollout producing network G with $|P^+|$ on-patterns in total,

$$r(G) = \begin{cases} w \frac{| \text{covered} |}{|P^+|}, & G \text{ incomplete,} \\ w + (1 - w) \frac{T_{\text{ref}}}{T_{\text{eff}}(G)}, & G \text{ complete,} \end{cases}$$

with $w \in (0, 1)$. Complete networks always out-score incomplete ones, and among complete networks the reward increases as the network shrinks.

Self-tightening reference. T_{ref} is *not* an externally supplied optimum: it is the smallest reduced size the policy has itself produced for this function so far. Matching one’s own record scores exactly 1, beating it scores above 1, and regressing scores below. As the policy improves, the bar rises automatically, so no oracle cost is needed during training.

6.2 Advantage Estimation

Let $r_1, \dots, r_K$ be the rewards of K rollouts drawn for the same target. The group-relative (critic-free) estimator uses the batch as its own baseline, $A_i = (r_i - \bar{r})/(\sigma_r + \varepsilon)$. This is simple but has a structural failure mode: once the policy converges so that all K rollouts produce the same network, $\sigma_r \rightarrow 0$ and $A_i \rightarrow 0$, so learning halts even if the converged network is suboptimal. Two remedies are used.

Absolute anchoring. Adding a term $\beta(r_i - 1)$ re-introduces an absolute scale, so a group that is uniformly worse than its own record receives uniformly negative advantage instead of none.

Table 1: Source-selection accuracy on 4-input functions (architecture $\times$ supervision order).

Architecture	Breadth-first order	Path order
Base	0.543	0.886
Wide + attention	0.625	0.932

Learned critic with dense credit. Alternatively, each placement is rewarded by its own contribution, $r_t = \alpha \Delta \text{coverage}_t - c_{\text{step}}$, with a completion bonus at the final step, and advantages are formed against a learned value function, $A_t = R_t - V_\phi(s_t)$. Because the baseline is a function of the state rather than of the batch, it does not degenerate when rollouts agree. A self-imitation term additionally replays the best trajectory found so far with positive-part advantage, consolidating rare discoveries.

6.3 Escaping Converged Local Optima

Even with a non-degenerate baseline, a policy that has converged one transistor above the optimum receives almost no gradient toward the optimum, because the reward difference $T_{\text{ref}}/T_{\text{eff}}$ between an n - and an $(n-1)$ -transistor solution is small, while the self-imitation term actively reinforces the incumbent. We use three countermeasures: a *novelty* bonus for producing a structurally unseen complete network, a discrete bonus for strictly beating the running record, and down-weighting self-imitation while the record remains above the target. A scheduler additionally re-heats the sampling temperature and entropy coefficient whenever the record stalls, and cools it again on improvement.

7. Experiments

7.1 Setup

We evaluate on exhaustive sweeps of 3- and 4-input Boolean functions and on a catalog of 6-input non-series-parallel (NSP) functions. Each function contributes two synthesis targets, the pull-down network realizing f and the pull-up network realizing $\neg f(\neg x)$. Reference costs are the minimum-transistor solutions from an exhaustive SAT-based sweep; a target counts as *solved* when the policy produces a correct network whose reduced size T_{eff} meets or beats that reference.

7.2 Construction Order Dominates Architecture

Table 1 isolates the effect of the supervision linearization of Section 3.3. Under an index-based breadth-first order, source-selection accuracy saturates near 0.5–0.6 regardless of model size; under the path decomposition it exceeds 0.88 with the same encoders. The ordering contributes roughly +30 accuracy points, while doubling model width contributes 4–8.

The same effect appears at 6 inputs, where accuracy rises from 0.505 to 0.91. We attribute this to the structural determinism argued in Section 3.3: under the path order the source is fixed by the graph during walk extensions, whereas under an index-based order it is an arbitrary convention the model must memorize.

7.3 Label Consistency

Retaining every implementation of a function in the supervision set introduces conflicting targets for identical states. On the 6-input sweep, restricting supervision to one implementation per

function raises source accuracy from 0.443 to 0.505 and, unlike the multi-implementation setting, the loss continues to decrease rather than degrading.

7.4 Transistor-Count Optimization

3-input functions. All 254 functions were synthesized for both polarities (508 targets). The pipeline solves 508/508 (100%) at the SAT-optimal cost. Notably 496 targets (97.6%) are already optimal in the pretrained policy’s first sampled evaluation, with only 12 requiring reinforcement learning, each converging within 12 updates.

4-input functions. Optimization over the full sweep is in progress; on the targets completed so far the pipeline reaches the SAT-optimal cost on 88%. A frozen random subset of 256 solved targets is retained as a fixed evaluation set for the physical-optimization experiments.

6-input NSP functions. On the 53-function NSP catalog, the pretrained policy alone produces a correct network for 50/53 functions, but at a mean reduced cost of 19.0 transistors against references near 7: completion is largely solved by pretraining, whereas compaction is the task left to reinforcement learning.

7.5 Diagnostic Analyses

Top- k behaviour. Although top-1 source accuracy is 0.50 on 6-input states, the correct node lies within the top 3 candidates 78% of the time and within the top 5 89% of the time (mean rank 1.6 of ~ 64). Because finetuning samples from the policy rather than taking its argmax, this is the relevant measure of prior quality.

Where source selection fails. Accuracy is 0.62 on small partial networks but falls to 0.36 on mid-sized ones, and is markedly lower when the target is an interior node (0.37) than a rail (0.66); the model over-predicts rails. Errors are generally not near-misses, and the model is confidently wrong rather than undecided, indicating a learning rather than a tie-breaking failure.

Representational ceiling. A Weisfeiler–Leman refinement over the state graphs assigns every candidate node a unique colour within two rounds, so the achievable source accuracy is 1.0: the observed gap is an optimization and inductive-bias limitation, not a representational one.

Cost of the reinforcement-learning variants. All reward variants cost essentially the same per update (≈ 1.6 s under identical conditions), since rollout generation dominates and the critic and exploration mechanisms are negligible in comparison. Throughput is instead governed by hardware sharing: per-update time grows from 1.9 s with one process per accelerator to 20 s at five and 49 s at sixteen, because each generation step issues many small kernels that serialize.

8. Planned Experiments

The following studies are planned to complete the evaluation.

Necessity of pretraining. Finetuning from a randomly initialized policy, versus from the pretrained policy, isolates the contribution of imitation pretraining and validates the two-stage design.

Component ablation of the escape mechanisms. Section 6.3 introduces four mechanisms together; a leave-one-out study (removing novelty, record-beating bonus, self-imitation down-weighting, and temperature re-heating in turn) is required to attribute the gain to individual components.

Variance across seeds. Per-target outcomes on hard instances are stochastic: variants have been observed to exchange wins on the same instance across runs. All comparisons will be repeated over multiple seeds and reported with dispersion rather than as single runs.

Reduction ablation. Reporting unreduced, topologically reduced, and functionally reduced costs side by side quantifies how much of the apparent gap to the reference is genuine redundancy.

Node features. The failure analysis localizes errors to interior nodes of mid-sized networks. Supplying per-node structural and functional descriptors (distance to the rails, degree, and the number of uncovered on-patterns routed through each node) targets exactly this deficit.

Search budget. Completion depends strongly on the step budget; a sweep over budgets separates failures caused by exhausting the budget from those caused by an inability to share structure.

Baselines. Comparison against the exhaustive SAT reference, a classical factoring-based synthesizer, and the naive sum-of-products series-parallel construction places the learned results on a common scale.

Physical objective. The same framework is being extended so that the reward is a placement-quality metric returned by a layout engine. Because that metric is only defined once both the pull-down and pull-up networks are complete, a target is a *function* rather than a single network, and the reward is assigned jointly to the two trajectories.